perplexity

1. NVIDIA Blackwell Hardware Decompression Engine & NVFP4 Streaming

1.1 Platform corrections that change the implementation

The proposed configuration combines capabilities that must not be assumed to coexist. The principal finding is **incompatibility**: NVIDIA's documented datacenter Blackwell DE path is not a documented GB10 capability, and neither ordinary RDMA verbs nor `io_uring` provide the completely kernel-free, polling-free synchronization path described in the request.

Item	Verified boundary
GB10 CUDA architecture	NVIDIA identifies GB10/DGX Spark as compute capability 12.1, not 12.0. Target `sm_121` and query device capabilities rather than importing B200 assumptions. Official mapping: https://developer.nvidia.com/cuda/gpus. [1]
Unified memory	DGX Spark documents 128 GB coherent LPDDR5X and 273 GB/s memory bandwidth. Physical shared memory is not equivalent to a `cudaMallocManaged` allocation or to arbitrary device-accessible virtual addresses. Hardware specification: https://docs.nvidia.com/dgx/dgx-spark/hardware.html. [2]
Network speed	NVIDIA's DGX Spark specification lists ConnectX-7 at 200 Gbps, not 400 Gbps. A different OEM/external 400-Gbps configuration requires its own PCIe topology and negotiated-link verification. https://www.nvidia.com/en-us/products/workstations/dgx-spark/. [3]
Hardware decompression	nvCOMP's DE FAQ lists B200, B300, GB200, and GB300. It does not list GB10. Therefore, 600 GB/s DE throughput cannot be assigned to GB10 from the Blackwell brand name. https://docs.nvidia.com/cuda/nvcomp/decompression_engine_faq.html. [4]

The distinction between SM families is material: NVIDIA documents different occupancy and shared-memory limits for compute capabilities 10.0 and 12.0. A B200 kernel/memory-pipeline design is not automatically a GB10 design. See Blackwell Tuning Guide, architecture and occupancy sections: https://docs.nvidia.com/cuda/blackwell-tuning-guide/index.html. [5]

1.2 DE support versus nvCOMP support

nvCOMP 4.2 introduced Blackwell DE integration. The library supports more algorithms than the fixed-function engine supports; these are separate compatibility lists. https://developer.nvidia.com/nvcomp. [6]

Algorithm	nvCOMP availability	Documented fixed-function DE support	Verified GB10 NVFP4-tensor DE benchmark
LZ4	Yes	Yes	Not established by the retrieved primary sources
Snappy	Yes	Yes	Not established

Algorithm	nvCOMP availability	Documented fixed-function DE support	Verified GB10 NVFP4-tensor DE benchmark
Deflate/Gzip	Yes	Yes	Not established
GDeflate	Yes	Not in the documented DE codec list	Not established
Bitcomp	Yes	Not in the documented DE codec list	Not established

Sources: nvCOMP algorithm documentation, https://docs.nvidia.com/cuda/nvcomp/index.html; DE FAQ, “What compression formats are supported by DE?” https://docs.nvidia.com/cuda/nvcomp/decompression_engine_faq.html. [4] [7]

NVIDIA advertises DE decompression “up to 600 GB/s.” That is not an empirical NVFP4-weight benchmark, not a guaranteed per-codec rate, not an NVMe-to-GEMM rate, and not a GB10 measurement. The official integration article is dated October 6, 2025: https://developer.nvidia.com/blog/speeding-up-data-decompression-with-nvcomp-and-the-nvidia-blackwell-decompression-engine/. [8]

No retrieved primary source establishes the requested matrix of:

- GB10/GB20B;
- actual NVFP4 or MXFP4 expert weights;
- Bitcomp/GDeflate/Snappy/LZ4;
- compression ratio and compressed-input/decompressed-output bandwidth;
- concurrent NVMe DMA and GEMM.

Reporting such numbers would fabricate evidence.

1.3 NVFP4 is an execution format, not a DE codec

The actual layouts are:

$$\text{NVFP4}: \quad \text{E2M1 values} + \text{E4M3 scale per 16 values} + \text{FP32 tensor scale}.$$

$$\text{MXFP4}: \quad \text{E2M1 values} + \text{E8M0 scale per 32 values}.$$

NVFP4 therefore costs approximately 4.5 bits/value before the tensor-level scalar and layout padding; MXFP4 approximately 4.25 bits/value. NVIDIA references: https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/ and https://research.nvidia.com/labs/eai/blogs/pushing-intelligence-to-4-bit/. [9] [10]

Consequently, “decompress NVFP4” must distinguish:

1. Lossless decompression of a byte stream containing already-quantized FP4 values and scales.
2. Quantization/dequantization and scale interpretation by the execution kernel.

nvCOMP addresses the first. It does not turn arbitrary compressed weights into a tensor-core-ready layout or replace FP4 scale handling. Its interfaces operate on compressed chunks;

NVFP4's interpretation belongs to the tensor execution format.[7] [9]

1.4 Exact allocation boundary and fail-closed selection

The current DE FAQ makes a more nuanced distinction than "all buffers require a special pool."[4]

Allocation	Documented behavior
`cudaMalloc`, `cuMemAlloc`	DE-compatible for device-to-device decompression
`cudaMallocPitch`	DE-compatible
`cudaMallocAsync`	SM fallback
`cudaMallocManaged`	SM fallback
`cudaHostAlloc`, `cudaMallocHost`	SM fallback
`cudaMallocFromPoolAsync`, ordinary pool	SM fallback
`cudaMallocFromPoolAsync`, `cudaMemPoolCreateUsageHwDecompress` pool	DE-compatible
`cuMemCreate`, `CU_MEM_CREATE_USAGE_HW_DECOMPRESS`	Specialized DE-compatible allocation path

Source: FAQ sections "I use cudaMalloc...," "Does DE work with host memory?" and allocation table. https://docs.nvidia.com/cuda/nvcomp/decompression_engine_faq.html.[4]

The relevant implementation pattern is capability validation plus explicit hardware backend selection—not inference from page pinning:

```
/* Compile as C against CUDA Driver API headers/library.
   This is a capability predicate, not a decompression operation. */
#include <cuda.h>
#include <stdint.h>

int buffer_supports_hw_decompression(const void *p)
{
    unsigned char capable = 0;
    CUresult r = cuPointerGetAttribute(
        &capable,
        CU_POINTER_ATTRIBUTE_IS_HW_DECOMPRESS_CAPABLE,
        (CUdeviceptr)(uintptr_t)p);
    return r == CUDA_SUCCESS && capable != 0;
}
```

For the chosen codec's `nvcompBatched<Format>DecompressOpts_t`, set:

```
opts.backend = NVCOMP_DECOMPRESS_BACKEND_HARDWARE;
```

`NVCOMP_DECOMPRESS_BACKEND_DEFAULT` permits silent SM fallback; `...HARDWARE` returns an error for incompatible allocations. Validate compressed buffers, output buffers, and relevant uncompressed-size storage; query `CU_DEVICE_ATTRIBUTE_MEM_DECOMPRESS_MAXIMUM_LENGTH` rather than hard-coding the documented B200 maximum of 4 MiB.[4]

Physical page pinning alone is insufficient: the FAQ explicitly shows page-locked host allocations falling back to SMs.[4]

1.5 PCIe concurrency, LPDDR5X, and TMEM

On supported datacenter devices, NVIDIA documents DE integration with the copy engine, compressed transfer over PCIe/C2C, and overlap with SM execution. That does not establish an SSD peer-DMA integration, a GB10 engine, or a guarantee of independent bandwidth under concurrent traffic.[4]

There is no documented nvCOMP destination allocator for TMEM in these sources. The API contract describes memory buffers, not an NVMe/RDMA-addressable TMEM aperture. Do not design around "NVMe → DE → TMEM"; the supported DE integration described here ends in compatible CUDA memory buffers.[4]

For GB10, the defensible pipeline is therefore:

```
NVMe → registered system-memory destination
     → optional measured SM decompression/layout conversion
     → FP4 execution kernel
```

Whether the optional compression stage helps must be measured on actual quantized weights. No GB10 hardware-DE throughput or concurrency number is established by the cited documentation.

2. State-of-the-Art in Predictive / Lookahead Routing for MoE

2.1 Literature map: do not conflate placement prediction, layer lookahead, and draft-token prediction

System	Exact reference	Relevant mechanism and boundary
ProMoE	arXiv:2410.22134v3, https://arxiv.org/html/2410.22134v3; code https://github.com/promoe-opensource/promoe	Learned intermediate-state predictor; stride prefetching increases prediction distance by one; precise-demand priority and chunked transfers. [11]
FATE	arXiv:2502.12224, https://arxiv.org/abs/2502.12224	Adjacent-layer gate-input reuse; training-free predictor. Reported cache hit rate must not be interpreted as predictor recall. [12]
SP-MoE	arXiv:2510.10302v2, https://arxiv.org/html/2510.10302v2	Draft-model attention outputs evaluated with corresponding target-model gates; cutoff-layer policy. This is draft/verification alignment, not simply $H = 3$ from a current target-layer hidden state. [13]

System	Exact reference	Relevant mechanism and boundary
Pre-attention prediction	arXiv:2511.10676, https://arxiv.org/abs/2511.10676	Two linear functions and ranking-aware loss, using pre-attention activations within the same layer. Not cross-layer $H = 2/3$. [14]
Director	arXiv:2607.08782, https://arxiv.org/abs/2607.08782	Cascaded predictor or low-bit replica for proactive distributed expert placement. Reported 11–55% latency reduction concerns placement optimization, not an $H = 1, 2, 3$ routing-recall table. [15]
SpecPrefetch	arXiv:2607.24787v2, https://arxiv.org/html/2607.24787v2	Low-rank next-layer predictor, frozen native router, KL distillation, bounded transfer budget. [16]
SPICE	arXiv:2608.21240v1, https://arxiv.org/html/2608.21240v1	Recursive multi-layer forecast, confidence-aware lookahead, resident low-rank expert surrogates, CPU exact recovery. [17]

2.2 Explicit predictor and loss: SpecPrefetch

Sections 3.2–3.4 give an unusually explicit router-preserving formulation. For hidden dimension D, n experts, and rank r:

$$\hat{z}^{l+1} = B^l A^l x^l, \qquad A^l \in \mathbb{R}^{r \times D}, \quad B^l \in \mathbb{R}^{n \times r}.$$

$$\hat{\pi}^{l+1} = \mathrm{softmax}(\hat{z}^{l+1}), \qquad s_e^{l+1} = \sum_{(b,s) \in \Omega} \hat{\pi}_{b,s,e}^{l+1}.$$

Training minimizes:

$$
[
\mathcal L_{\rm pred}
$$

\frac1{L-1}\sum_l
\frac1{|\Omega|}
\sum_{(b,s)\in\Omega}
\operatorname{KL}
\left(
\operatorname{stopgrad}(\pi_{b,s}^{l+1})
;|;
\hat\pi_{b,s}^{l+1}
\right).
]

The target model stays frozen. Predicted experts control transfers only; the native router remains authoritative. https://arxiv.org/html/2607.24787v2, §§3.2–3.4. [16]

The runtime excludes resident and already-in-flight experts, then transfers the highest-priority remaining candidates under $M_{\max}$. A true miss loads on demand rather than skipping the native expert. [16]

2.3 Over-provisioning: verified recall numbers

SpecPrefetch Table 1 reports the following next-layer results; these are $H = 1$, not $H = 2/3$.[16]

Model / workload	Native k	Recall at $M = k$	Recall at $M = k + 2$
Qwen3-VL-30B-A3B / GSM8K	8	85.60%	92.31%
Qwen3-VL-30B-A3B / HumanEval	8	85.48%	92.16%
Qwen3-VL-30B-A3B / ChartQA	8	89.40%	95.48%
DeepSeek-VL2-Tiny / GSM8K	6	84.35%	92.47%
DeepSeek-VL2-Tiny / ChartQA	6	89.61%	96.83%

The same paper reports that throughput peaks at a bounded prefetch budget and declines with larger budgets despite improved readiness. On its Snapdragon platform, executing prediction/control without transfers changes throughput from 3.753 ± 0.164 to 3.730 ± 0.113 tokens/s; enabling prefetch produces 4.055 ± 0.129 tokens/s. These are mobile runtime measurements, not GB10 FLOP overhead measurements. https://arxiv.org/html/2607.24787v2, Tables 4–5 and §4.4.[16]

2.4 Multi-layer prediction: SPICE's actual formulation

SPICE §4.1 recursively rolls forward a lightweight state:

$$
[
\hat{\mathcal S}^{l+\delta}
\operatorname{TopK}(\hat\pi^{l+\delta},K),
]
$$

$$
[
\hat h^{j}
\operatorname{Attn}^{j}(\hat h^{j-1})
+
\sum_{i\in\hat{\mathcal S}^{j}}
\hat\pi_i^j,\tilde E_i^j(\hat h^{j-1}).
]
$$

Its overlap-derived minimum horizon is:

$$
H_{\min}
$$

\left\lceil
\frac{P M_e}
{B_{\rm PCIe}(T_{\rm attn}+K T_{\rm expert})}
\right\rceil.
]

After reaching that depth, speculation stops when the selected experts' predicted probability mass falls below a threshold:

$$
\mathcal C(\hat\pi^j)
$$

\sum_{i\in\hat{\mathcal S}^{{j}}\hat\pi_i<\tau,
]

or when $H_{\max}$ is reached. https://arxiv.org/html/2608.21240v1, §4.1, Eqs. 2–5 and Algorithm 1.[17]

Important implementation detail: maintain prefetch keys as `(layer, expert)`, not just expert IDs. Expert number 7 in layer $l+1$ is not expert number 7 in layer $l+2$.

SPICE's recovery surrogate is:

$$\tilde{E}_e(h) = S(h) + B_e A_e h,$$

trained with:

$$\min_{A_e, B_e} \sum_{h \in \mathcal{D}_{l,e}} \| E_e(h) - S(h) - B_e A_e h \|_2^2.$$

Low-confidence misses can use this approximation; other misses use exact CPU execution or GPU fetch. The approximation is not semantics-preserving: reported quality losses include 3.0–3.5 percentage points on GSM8K and 2.3–3.9 points on HumanEval.
https://arxiv.org/html/2608.21240v1, §4.2 and Table 1.[17]

2.5 What the evidence does—and does not—provide for $H = 1, 2, 3$

ProMoE reports:

- Mean learned-predictor accuracy of 84.7% in its evaluation.
- Approximately 200 μs CPU predictor latency.
- A reported "5%" accuracy decline when stride prefetching increases prediction distance by one.

The text does not justify converting that last statement into an exact percentage-point $H = 2$ recall value, and it does not establish $H = 3$. https://arxiv.org/html/2410.22134v3, §§4.3–4.4.[11]

Neither the retrieved SPICE text nor the other cited results supplies a controlled, directly comparable table of exact expert recall at $H = 1, 2, 3$ with identical model, dataset, cache, and candidate count. That requested core datum remains unverified.

A useful analytical FLOP accounting—not an empirical paper result—is:

$$F_{\text{lowrank}} \approx 2r(D + n)$$

per token per predicted target layer, excluding softmax/top-M. Relative to a three-projection gated FFN with intermediate width d_{ff},

$$F_{\text{FFN}} \approx 6Dd_{\text{ff}}, \qquad \frac{F_{\text{lowrank}}}{F_{\text{FFN}}} \approx \frac{r(D + n)}{3Dd_{\text{ff}}}.$$

This follows from SpecPrefetch's published matrix dimensions. It must not be applied to SPICE's full attention/surrogate rollout, whose work is different. [17] [16]

2.6 Miss handling suitable for an exact engine

ProMoE provides concrete scheduling mechanics:

1. Predict into a low-priority queue.
2. Split expert transfers into chunks.
3. Once the native gate completes, remove obsolete speculative requests.
4. Promote true misses into a high-priority queue.
5. Execute already-resident experts while missing experts arrive.

Its implementation naturally splits a three-projection expert into three chunks. An issued CUDA copy cannot be preempted halfway; scheduling preemption occurs between chunks. https://arxiv.org/html/2410.22134v3, §§5.1–5.4. [11]

For an exact inference engine, distinguish cancellation of incorrect prefetches from skipping router-selected experts. The former preserves model execution; the latter generally does not. SpecPrefetch implements the former separation explicitly. [16]

3. Pure C / Bare-Metal Zig Zero-Copy RDMA (RoCE v2) Bypassing the Kernel

3.1 What is actually possible

Userspace verbs bypass the kernel's network data path after setup; they do not eliminate the kernel driver, MR registration, QP creation, or all completion-notification overhead. The maintained implementation is rdma-core: https://github.com/linux-rdma/rdma-core. NVIDIA's 2026 guide contains RC QP setup and operation examples: https://networking-docs.nvidia.com/doca/archive/3-5-0/rdma-aware-networks-programming-guide. [18] [19]

Three requirements must be separated:

Requirement	Actual boundary
No NCCL/MPI	Achievable with libibverbs RC QPs
No OS TCP/IP payload stack	Achievable with RoCE hardware transport
No polling and no kernel transitions	Not delivered by standard CPU-side verbs completion handling; event notification avoids busy-polling but uses completion-channel events

`RDMA_WRITE_WITH_IMM` generates a receiver CQE containing the 32-bit immediate value. It does not automatically increment an arbitrary application-memory counter. https://www.man7.org/linux/man-pages/man3/ibv_wr_rdma_write_imm.3.html. [20]

3.2 DeepEP is not a pure-C drop-in

The current DeepEP repository uses NCCL Gin, C++20, GPU kernels, and SM resources for EP dispatch/combine. Its current README explicitly states that zero-SM RDMA EP is not supported. It also says InfiniBand is fully tested while RoCE is theoretically compatible. Those qualifications prohibit presenting it as verified, production-grade GB10 RoCE pure-C code. https://github.com/deepseek-ai/DeepEP. [21]

A useful established setup reference is rdma-core's RC example:

https://github.com/linux-rdma/rdma-core/blob/master/libibverbs/examples/rc_pingpong.c. [22]

For a no-TCP bootstrap, exchange QPN, PSN, RoCEv2 GID/GID index, remote virtual address, and rkey through a launcher-provided manifest. The QPs still require correct INIT → RTR → RTS transitions; the NVIDIA guide documents this setup. [19]

3.3 Concrete C registration, descriptors, and event-driven completion

The following is a self-contained, compilable C translation unit implementing the requested primitives. It intentionally takes an already-connected RC QP: discovery, fabric addressing, authentication, and QP connection are control-plane responsibilities, not hidden behind an invented "bare-metal" transport.

Compile:

```
cc -std=c11 -O2 -Wall -Wextra -c moe_ring.c
```

Link the resulting object with `-libverbs`. The API patterns follow the maintained verbs interfaces and NVIDIA RC programming guide. [23] [19] [20]

```
#define _POSIX_C_SOURCE 200112L
#include <infiniband/verbs.h>
#include <arpa/inet.h>
#include <stdatomic.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
```

```
#include <unistd.h>

enum { NSLOT = 256, SLOT_BYTES = 65536 };

struct token_ring {
    unsigned char *base;
    struct ibv_mr *mr;
    _Atomic uint64_t ready[NSLOT];
};

/* Allocate ordinary system RAM; ibv_reg_mr pins/registers it.
   GPU accessibility is a separate CUDA/platform contract. */
int ring_create(struct token_ring *r, struct ibv_pd *pd)
{
    memset(r, 0, sizeof(*r));
    long pg = sysconf(_SC_PAGESIZE);
    if (pg <= 0) return EINVAL;

    int e = posix_memalign((void **)&r->base,
                           (size_t)pg,
                           (size_t)NSLOT * SLOT_BYTES);
    if (e) return e;

    memset(r->base, 0, (size_t)NSLOT * SLOT_BYTES);
    for (unsigned i = 0; i < NSLOT; ++i)
        atomic_init(&r->ready[i], 0);

    r->mr = ibv_reg_mr(pd, r->base,
                       (size_t)NSLOT * SLOT_BYTES,
                       IBV_ACCESS_LOCAL_WRITE |
                       IBV_ACCESS_REMOTE_WRITE);
    if (!r->mr) {
        e = errno ? errno : EIO;
        free(r->base);
        r->base = NULL;
        return e;
    }
    return 0;
}

/* WRITE_WITH_IMM consumes a receive WQE although payload placement
   uses remote_addr/rkey, not the receive SGE. */
int post_notification_credit(struct ibv_qp *qp, uint64_t id)
{
    struct ibv_recv_wr wr = {0}, *bad = NULL;
    wr.wr_id = id;
    return ibv_post_recv(qp, &wr, &bad);
}

/* Immediate layout: [31:24] layer, [23:16] expert, [15:8] epoch,
   [7:0] slot. Full generation/length belongs in the slot header. */
int post_token_write(struct ibv_qp *qp,
                     struct token_ring *src,
                     unsigned src_slot,
                     unsigned dst_slot,
                     uint64_t remote_base,
```

```
                     uint32_t remote_rkey,
                     uint32_t bytes,
                     uint8_t layer,
                     uint8_t expert,
                     uint8_t epoch,
                     uint64_t operation_id)
{
    if (src_slot >= NSLOT || dst_slot >= NSLOT ||
        !bytes || bytes > SLOT_BYTES)
        return EINVAL;

    uint32_t meta = ((uint32_t)layer << 24) |
                    ((uint32_t)expert << 16) |
                    ((uint32_t)epoch << 8) | dst_slot;

    struct ibv_sge sge = {
        .addr = (uintptr_t)(src->base +
                            (size_t)src_slot * SLOT_BYTES),
        .length = bytes,
        .lkey = src->mr->lkey
    };
    struct ibv_send_wr wr = {0}, *bad = NULL;
    wr.wr_id = operation_id;
    wr.sg_list = &sge;
    wr.num_sge = 1;
    wr.opcode = IBV_WR_RDMA_WRITE_WITH_IMM;
    wr.send_flags = IBV_SEND_SIGNALED;
    wr.imm_data = htonl(meta);
    wr.wr.rdma.remote_addr =
        remote_base + (uint64_t)dst_slot * SLOT_BYTES;
    wr.wr.rdma.rkey = remote_rkey;

    return ibv_post_send(qp, &wr, &bad);
}

/* One CPU thread owns this receive CQ and QP receive queue.
   CQ must initially be armed using ibv_req_notify_cq(cq, 0).
   The event channel must be blocking. */
int receive_event_batch(struct ibv_comp_channel *channel,
                        struct ibv_qp *qp,
                        struct token_ring *dst)
{
    struct ibv_cq *cq;
    void *context;
    if (ibv_get_cq_event(channel, &cq, &context))
        return errno ? errno : EIO;

    ibv_ack_cq_events(cq, 1);

    /* Re-arm before draining: avoids the drain/arm lost-wakeup race. */
    if (ibv_req_notify_cq(cq, 0))
        return errno ? errno : EIO;

    struct ibv_wc wc[^32];
    int n;
    while ((n = ibv_poll_cq(cq, 32, wc)) > 0) {
```

```
        for (int i = 0; i < n; ++i) {
            if (wc[i].status != IBV_WC_SUCCESS)
                return EIO;
            if (wc[i].opcode != IBV_WC_RECV_RDMA_WITH_IMM ||
                !(wc[i].wc_flags & IBV_WC_WITH_IMM))
                return EPROTO;

            uint32_t m = ntohl(wc[i].imm_data);
            unsigned slot = m & 255u;

            /* Publish only after CQE observation. This counter is
               incremented by this CPU, not by the immediate field. */
            atomic_fetch_add_explicit(&dst->ready[slot], 1,
                                      memory_order_release);

            int e = post_notification_credit(qp, wc[i].wr_id);
            if (e) return e;
        }
    }
    return n < 0 ? EIO : 0;
}

uint64_t slot_ready_generation(struct token_ring *r, unsigned slot)
{
    return atomic_load_explicit(&r->ready[slot],
                                memory_order_acquire);
}

int ring_destroy(struct token_ring *r)
{
    /* Caller must first drain all operations referencing the MR. */
    if (r->mr && ibv_dereg_mr(r->mr))
        return errno ? errno : EIO;
    free(r->base);
    r->base = NULL;
    r->mr = NULL;
    return 0;
}
```

The immediate-data delivery and event-channel semantics are specified by the verbs API: https://www.man7.org/linux/man-pages/man3/ibv_wr_rdma_write_imm.3.html and https://www.man7.org/linux/man-pages/man3/ibv_get_cq_event.3.html. [20] [23]

3.4 Correctness conditions the descriptor alone cannot solve

- Provision receive-notification WQEs before sending writes with immediate.
- Maintain separate payload-slot credits. Reposting a notification WQE does not authorize overwriting a payload still being consumed.
- Reuse a source slot only after the relevant send completion.
- Use a full generation number in each payload header; an 8-bit immediate epoch wraps.
- Partition destination slots per source rank or implement an explicit allocator.

- For combine, write each expert contribution to a distinct slot and reduce locally; overlapping ordinary RDMA writes do not implement a floating-point sum.

The WR/CQE lifecycle is grounded in the RC programming guide; the ring layout and credit protocol above are implementation choices, not a claim that verbs supplies an MoE collective. [19]

3.5 CPU publication is not GPU publication

An acquire/release pair synchronizes CPU threads. It is not sufficient proof that an already-running GPU kernel can safely consume third-party DMA writes.

NVIDIA's GPUDirect RDMA guide explicitly warns that running kernels can observe stale or out-of-order data and requires the appropriate CUDA synchronization/work-submission ordering. https://docs.nvidia.com/cuda/gpudirect-rdma/, memory-ordering and synchronization discussion. [24]

Thus:

```
NIC CQE observed
→ CPU publication to CPU consumers
→ separately established CUDA visibility/order
→ GPU consumer
```

Do not label `atomic_thread_fence()` an RDMA-to-GPU synchronization primitive. A no-CPU-progress GPU consumer requires a platform-supported GPU/NIC signaling protocol and its documented memory-ordering contract, not merely a 32-bit immediate.

4. Maximum I/O Yields via io_uring / O_DIRECT into Unified Memory

4.1 Verified benchmark data and its scope

The strongest retrieved 2024–2026 primary benchmark is:

Matthias Jasny et al., "io_uring for High-Performance DBMSs: When and How to Use It," arXiv:2512.04859v1, https://arxiv.org/html/2512.04859v1.

Artifact: https://github.com/mjasny/vldb26-iouring. [25]

Measurement	Reported result	Qualification
Sequential read saturation	Up to 90 GiB/s	Eight PCIe 5.0 Kioxia CM7-R SSDs; not one SSD
Sequential write saturation	Up to 50 GiB/s	Same array
Read request size at saturation	256 KiB	With passthrough and IOPOLL
Write request size at saturation	128 KiB	Same optimized path
Single-read latency used in modeling	70 µs	Storage-engine device/workload context; not large-block sequential tail latency

Measurement	Reported result	Qualification
SQPOLL wakeup overhead	Approximately 30 µs	Dedicated microbenchmark; not SSD media latency

Source: §§2.2, 3.2, 3.4, and 3.6.[25]

These results do not establish:

- A single Gen5 x4 SSD's maximum throughput into GB10 LPDDR5X.
- NVFP4 tensor-loading latency.
- An equivalent 90 GiB/s result using ordinary fixed-buffer filesystem reads.
- A concurrent GB10 memory-bound GEMM interference curve.

The array benchmark uses additional optimizations, especially NVMe passthrough and IOPOLL. It must not be relabeled as a stock `O_DIRECT + SQPOLL` single-drive result.[25]

4.2 Exact API and alignment boundaries

The relevant structs and operations are:

```
struct io_uring_params
struct io_uring
struct iovec
struct io_uring_sqe
struct io_uring_cqe

io_uring_queue_init_params()
io_uring_register_buffers()
io_uring_register_files()
io_uring_prep_read_fixed()
IOSQE_FIXED_FILE
```

Maintained source: https://github.com/axboe/liburing; fixed-read contract: https://man7.org/linux/man-pages/man3/io_uring_prep_read_fixed.3.html. The selected buffer and byte range must fall inside the registered region identified by `buf_index`.[26] [27]

`SQPOLL` creates a kernel polling thread. It can avoid most submission syscalls while awake; it does not remove the kernel storage stack or guarantee zero kernel transitions. Sleeping-thread wakeups and blocking completion waits still matter. https://github.com/axboe/liburing/blob/master/man/io_uring_setup.2 and arXiv:2512.04859 §2.2.[28] [25]

4.3 Functional C fixed-buffer direct-I/O loop

This standalone reader:

- Uses `O_DIRECT`.
- Requests SQPOLL.
- Registers a fixed file and 64 fixed buffers.

- Uses 256-KiB requests.
- Queries direct-I/O alignment through `STATX_DIOALIGN`.
- Measures completed bytes and application-observed submit-to-reap latency.
- Refuses misaligned file lengths instead of silently issuing an invalid tail read.
- Does not claim to validate CUDA accessibility.

It implements the maintained liburing fixed-buffer interface, not the paper's NVMe passthrough path. [27] [26]

```
#define _GNU_SOURCE
#include <liburing.h>
#include <linux/stat.h>
#include <sys/stat.h>
#include <sys/uio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <time.h>

enum { QD = 64, BS = 256 * 1024 };

static void die(const char *s)
{
    perror(s);
    exit(1);
}

static void ck(int r, const char *s)
{
    if (r < 0) {
        errno = -r;
        die(s);
    }
}

static uint64_t ns(void)
{
    struct timespec t;
    if (clock_gettime(CLOCK_MONOTONIC, &t)) die("clock_gettime");
    return (uint64_t)t.tv_sec * 1000000000ULL + t.tv_nsec;
}

static size_t maxz(size_t a, size_t b)
{
    return a > b ? a : b;
}

static size_t power2_ceil(size_t x)
{
```

```
    size_t p = sizeof(void *);
    while (p < x) {
        if (p > SIZE_MAX / 2) {
            errno = EOVERFLOW;
            die("alignment");
        }
        p *= 2;
    }
    return p;
}

int main(int argc, char **argv)
{
    if (argc != 2) {
        fprintf(stderr, "usage: %s aligned-weight-file\n", argv[^0]);
        return 2;
    }

    int fd = open(argv[^1], O_RDONLY | O_DIRECT | O_CLOEXEC);
    if (fd < 0) die("open");

    struct stat st;
    if (fstat(fd, &st)) die("fstat");
    if (!S_ISREG(st.st_mode) || st.st_size <= 0) {
        fprintf(stderr, "requires a nonempty regular file\n");
        return 2;
    }

    struct statx sx = {0};
    if (statx(fd, "", AT_EMPTY_PATH,
              STATX_SIZE | STATX_DIOALIGN, &sx))
        die("statx");

    if (!(sx.stx_mask & STATX_DIOALIGN) ||
        !sx.stx_dio_mem_align || !sx.stx_dio_offset_align) {
        fprintf(stderr, "filesystem did not report DIO alignment\n");
        return 2;
    }

    long pg = sysconf(_SC_PAGESIZE);
    if (pg <= 0) {
        errno = EINVAL;
        die("page size");
    }

    size_t align = power2_ceil(
        maxz((size_t)pg, sx.stx_dio_mem_align));
    size_t off_align = sx.stx_dio_offset_align;

    if (align % sx.stx_dio_mem_align ||
        BS % off_align ||
        (uint64_t)st.st_size % off_align) {
        fprintf(stderr, "file/request alignment unsupported\n");
        return 2;
    }
```

```
    struct iovec bufs[QD];
    for (unsigned i = 0; i < QD; ++i) {
        int e = posix_memalign(&bufs[i].iov_base, align, BS);
        if (e) {
            errno = e;
            die("posix_memalign");
        }
        bufs[i].iov_len = BS;
        memset(bufs[i].iov_base, 0, BS);
    }

    struct io_uring ring;
    struct io_uring_params p = {0};
    p.flags = IORING_SETUP_SQPOLL;
    p.sq_thread_idle = 2000;
    ck(io_uring_queue_init_params(QD, &ring, &p), "ring init");
    ck(io_uring_register_buffers(&ring, bufs, QD),
       "register buffers");
    ck(io_uring_register_files(&ring, &fd, 1),
       "register files");

    uint64_t started[QD] = {0};
    unsigned requested[QD] = {0};
    unsigned char busy[QD] = {0};
    uint64_t next = 0, done = 0, operations = 0;
    uint64_t latency_sum = 0, latency_max = 0;
    unsigned inflight = 0;
    const uint64_t length = (uint64_t)st.st_size;
    uint64_t begin = ns();

    while (done < length) {
        unsigned prepared = 0;

        for (unsigned i = 0; i < QD && next < length; ++i) {
            if (busy[i]) continue;
            struct io_uring_sqe *sqe = io_uring_get_sqe(&ring);
            if (!sqe) break;

            unsigned bytes = length - next < BS
                           ? (unsigned)(length - next) : BS;

            io_uring_prep_read_fixed(
                sqe, 0, bufs[i].iov_base, bytes, next, i);
            sqe->flags |= IOSQE_FIXED_FILE;
            io_uring_sqe_set_data64(sqe, i);

            started[i] = ns();
            requested[i] = bytes;
            busy[i] = 1;
            next += bytes;
            ++inflight;
            ++prepared;
        }

        /* Publish all prepared SQEs. SQPOLL consumes them.
           submit() also handles required SQ-thread wakeups. */
```

```
        if (prepared) {
            while (io_uring_sq_ready(&ring))
                ck(io_uring_submit(&ring), "submit");
        }

        if (!inflight) {
            fprintf(stderr, "no progress\n");
            return 1;
        }

        struct io_uring_cqe *cqe;
        ck(io_uring_wait_cqe(&ring, &cqe), "wait");

        do {
            uint64_t id = io_uring_cqe_get_data64(cqe);
            if (id >= QD || !busy[id]) {
                fprintf(stderr, "invalid completion\n");
                return 1;
            }
            if (cqe->res < 0) {
                errno = -cqe->res;
                die("read completion");
            }
            if ((unsigned)cqe->res != requested[id]) {
                fprintf(stderr, "short direct read: %d / %u\n",
                        cqe->res, requested[id]);
                return 1;
            }

            uint64_t elapsed = ns() - started[id];
            latency_sum += elapsed;
            if (elapsed > latency_max) latency_max = elapsed;
            ++operations;
            done += (unsigned)cqe->res;

            /*
              Consume or hand off this slot here.
              Production must wait for its final GPU/RDMA consumer
              before clearing busy[id].
            */
            busy[id] = 0;
            --inflight;
            io_uring_cqe_seen(&ring, cqe);
        } while (!io_uring_peek_cqe(&ring, &cqe));
    }

    double seconds = (ns() - begin) * 1e-9;
    printf("bytes=%llu seconds=%.6f GB/s=%.3f GiB/s=%.3f\n",
           (unsigned long long)done, seconds,
           done / seconds / 1e9,
           done / seconds / 1073741824.0);
    printf("submit-to-reap mean_us=%.3f max_us=%.3f\n",
           latency_sum / (double)operations / 1000.0,
           latency_max / 1000.0);

    ck(io_uring_unregister_files(&ring), "unregister files");
```

```
    ck(io_uring_unregister_buffers(&ring), "unregister buffers");
    io_uring_queue_exit(&ring);
    for (unsigned i = 0; i < QD; ++i) free(bufs[i].iov_base);
    close(fd);
    return 0;
}
```

Build and run:

```
cc -std=c11 -O3 -Wall -Wextra weight_read.c -luring -o weight_read
./weight_read weights.aligned
```

This is a storage benchmark with reusable slots, not a persistent multi-gigabyte expert cache. In an engine, the completion handler must retain slot ownership through decompression and GPU consumption before allowing overwrite.

4.4 Zero-copy has three separate contracts

Contract	What establishes it
Avoid filesystem page-cache staging	Direct-I/O path and its alignment requirements
Avoid repeated I/O registration work	Registered buffers plus fixed-buffer operations
Avoid a subsequent host-to-GPU copy	Separately verified CUDA accessibility, residency, and synchronization for that allocation

Registered `iovec` buffers establish the liburing contract; they do not establish the CUDA or nvCOMP DE contracts. In particular, NVIDIA explicitly documents `cudaMallocManaged` as an SM-fallback allocation for DE. [27] [4]

A shared LPDDR5X physical pool does not permit treating arbitrary CUDA device pointers as Linux direct-I/O buffers. Conversely, successful direct I/O into `posix_memalign` memory does not itself prove that a GPU kernel may dereference that CPU virtual address. The implementation must establish both interfaces independently.

4.5 LPDDR5X contention: measured evidence versus a bandwidth model

The official GB10 memory figure is 273 GB/s. No retrieved primary benchmark quantifies GB10 NVMe DMA interference with memory-bound FP4 GEMM under the requested fixed-buffer configuration. https://docs.nvidia.com/dgx/dgx-spark/hardware.html. [2]

A traffic model can guide measurement, but is not a benchmark. Let:

- C: compressed input bytes;
- U: decompressed output bytes.

A staged read/decompression/consume path can generate approximately:

$$
[
C_{\rm NVMe\ write}
+
C_{\rm decompressor\ read}
+
U_{\rm decompressor\ write}
+
U_{\rm GEMM\ read}
$$

2C+2U
]

of DRAM traffic, before activations, KV cache, scales, cache effects, and writebacks. An uncompressed read followed by weight consumption has approximately $2U$ of corresponding traffic. This accounting explains why reducing SSD bytes does not necessarily reduce shared-memory traffic.

For publishable GB10 results, collect a four-way matrix:

Experiment	Required outputs
NVMe only	Completed GB/s; mean/p50/p95/p99 submit-to-reap latency; CPU utilization
GEMM only	Kernel latency and achieved memory throughput
NVMe + GEMM	Both rates; GEMM slowdown; I/O latency inflation
NVMe + SM decompression + GEMM	Compressed-input GB/s, output GB/s, compression ratio, SM occupancy, end-to-end expert-ready latency

Record the actual SSD model, negotiated PCIe speed/width, firmware, filesystem, kernel, queue depth, request size, CUDA/nvCOMP versions, power/thermal state, and NIC traffic. Without that experiment, neither a B200 DE headline nor a multi-drive x86 `io_uring` benchmark supplies the missing GB10 execution-memory result.

⁂

1. https://developer.nvidia.com/cuda/gpus
2. https://docs.nvidia.com/dgx/dgx-spark/hardware.html
3. https://www.nvidia.com/en-us/products/workstations/dgx-spark/
4. https://docs.nvidia.com/cuda/nvcomp/decompression_engine_faq.html
5. https://docs.nvidia.com/cuda/blackwell-tuning-guide/index.html
6. https://developer.nvidia.com/nvcomp
7. https://docs.nvidia.com/cuda/nvcomp/index.html

8. https://developer.nvidia.com/blog/speeding-up-data-decompression-with-nvcomp-and-the-nvidia-blackwell-decompression-engine/
9. https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/
10. https://research.nvidia.com/labs/eai/blogs/pushing-intelligence-to-4-bit/
11. https://arxiv.org/html/2410.22134v3
12. https://huggingface.co/papers/2502.12224
13. https://arxiv.org/html/2510.10302v2
14. https://arxiv.org/abs/2511.10676
15. https://arxiv.org/abs/2607.08782
16. https://arxiv.org/html/2607.24787v2
17. https://arxiv.org/html/2608.21240v1
18. https://github.com/linux-rdma/rdma-core
19. https://networking-docs.nvidia.com/doca/archive/3-5-0/rdma-aware-networks-programming-guide
20. https://www.man7.org/linux/man-pages/man3/ibv_wr_rdma_write_imm.3.html
21. https://github.com/deepseek-ai/DeepEP
22. https://github.com/linux-rdma/rdma-core/blob/master/libibverbs/examples/rc_pingpong.c
23. https://www.man7.org/linux/man-pages/man3/ibv_get_cq_event.3.html
24. https://docs.nvidia.com/cuda/gpudirect-rdma/
25. https://arxiv.org/html/2512.04859v1
26. https://github.com/axboe/liburing
27. https://man7.org/linux/man-pages/man3/io_uring_prep_read_fixed.3.html
28. https://github.com/axboe/liburing/blob/master/man/io_uring_setup.2
29. https://platform.softwareone.com/product/nvidia-nvcomp/PCP-7454-6414
30. https://github.com/axboe/liburing/issues/1228
31. https://www.alphaxiv.org/abs/2511.10676
32. https://forums.developer.nvidia.com/t/speeding-up-data-decompression-with-nvcomp-and-the-nvidia-blackwell-decompression-engine/346966
33. https://www.linkedin.com/posts/talentsourcerrobkemp_nvidiablackwell-activity-7393317434604249088-ZAyz
34. https://github.com/MoE-Inf/awesome-moe-inference/
35. https://github.com/deepseek-ai/DeepEP/issues/155
36. https://www.marginalia.nu/log/a_123_index_io/
37. https://rtxsparks.com/guides/nvidia-dgx-spark-specs
38. https://qscompute.com/blog/nvidia-dgx-spark-gb10-local-ai-2026
39. https://deepwiki.com/NVIDIA/nvcomp/1.1-supported-compression-algorithms
40. https://tokenstead.ai/hardware/46
41. https://deepwiki.com/NVIDIA/nvcomp/5.2-algorithm-performance-comparisons
42. https://www.phoronix.com/reviews/Storage

43. https://www.newegg.com/insider/pcie-5-0-nvme-ssd-buying-guide-2026-is-it-worth-the-upgrade
44. https://docs.nvidia.com/cuda/pdf/GPUDirect_RDMA.pdf
45. https://developer.nvidia.com/gpudirect
46. https://docs.redhat.com/en/documentation/openshift_container_platform/4.19/html/hardware_accelerators/rdma-remote-direct-memory-access
47. https://resources.nvidia.com/en-us-accelerated-networking-resource-library/networking-solution
48. https://arxiv.org/html/2509.23638v2
49. https://arxiv.org/html/2509.07379v1
50. https://github.com/ali-vilab/ProMoE
51. https://arxiv.org/html/2510.26730v1
52. https://stackoverflow.com/questions/69534739/rdma-read-and-write-data-placement-visibility-semantics
53. https://github.com/axboe/liburing/wiki/io_uring-and-networking-in-2023
54. https://github.com/axboe/liburing/blob/master/src/include/liburing.h
55. https://newreleases.io/project/github/linux-rdma/rdma-core/release/v17
56. https://thegeekinthecorner.wordpress.com/2012/12/19/basic-flow-control-for-rdma-transfers/
57. https://unixism.net/loti/tutorial/fixed_buffers.html
58. https://github.com/axboe/liburing/issues/385
59. https://docs.rs/axboe-liburing
60. https://thegeekinthecorner.wordpress.com/category/infiniband-verbs-rdma/
61. https://networking-docs.nvidia.com/rdmaawareprogramming/
62. https://enterprise-support.nvidia.com/s/article/rdma-aware-networks-programming--160--user-manual
63. https://network.nvidia.com/products/GPUDirect-RDMA
64. https://docs.nvidia.com/rdma-aware-networks-programming-user-manual-1-7.pdf
65. https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/compute-capabilities.html
66. https://docs.nvidia.com/cuda//pdf/Blackwell_Tuning_Guide.pdf
67. https://docs.nvidia.com/doca/archive/doca-v2.2.0/rdma-programming-guide/index.html
68. https://developer.nvidia.com/blog/nvfp4-trains-with-precision-of-16-bit-and-speed-and-efficiency-of-4-bit/
69. https://learn-kernels.com/chapters/hardware/nvidia-blackwell
70. https://gpusmith.com/articles/en/cuda-compute-capability-table-gpu
71. https://arxiv.org/html/2603.08713v1
72. https://arxiv.org/html/2504.11651v2
73. https://developer.nvidia.com/cuda-toolkit-archive
74. https://developer.nvidia.com/cuda/toolkit
75. https://arxiv.org/pdf/2307.03760.pdf
76. https://developer.nvidia.com/cuda-downloads
77. https://arxiv.org/pdf/2312.10301.pdf
78. https://arxiv.org/abs/1107.1525

79. https://arxiv.org/abs/2111.09219

80. https://forums.developer.nvidia.com/t/gb10-and-docling/360665

81. https://forums.developer.nvidia.com/t/effective-pytorch-and-cuda/348230

82. https://github.com/Dao-AILab/flash-attention/issues/1969

83. https://liu-km.github.io/publication/2026-02-11-DIRECTOR-Accelerating-Distributed-MoE-Serving-via-Online-Proactive-Expert-Placement

84. https://www.aimodels.fyi/papers/arxiv/director-accelerating-distributed-moe-serving-via-online